

Escenario empresarial

Una cadena de tiendas minoristas necesita integrar ventas, inventario, CRM y analítica web en una tabla unificada de hechos de ventas. El reto es que las fuentes son heterogéneas (SQL, API, CSV, JSON) y los datos están fragmentados.

1. Identificación de fuentes de datos

- POS (SQL):
 - Tablas: ventas, productos ,tiendas
 - Contiene: id_venta, fecha/hora, id_tienda, id_producto, cantidad, precio_unitario
- Inventario (API REST):
 - Endpoint: /inventory/products
 - Contiene: stock_actual, id_producto, id_tienda
- CRM (CSV):
 - Archivos: clientes.csv
 - Contiene: id_cliente, nombre, segmento_cliente
- Web logs (JSON):
 - Archivos: logs.json
 - Contiene: id_cliente, timestamp, acción, producto_consultado

2. Esquema destino (tabla de hechos)

```
CREATE TABLE ventas_consolidadas (  
  id_venta INTEGER PRIMARY KEY,  
  fecha_venta DATE,  
  id_tienda INTEGER,  
  id_cliente INTEGER,  
  id_producto INTEGER,  
  cantidad INTEGER,  
  precio_unitario DECIMAL(10,2),  
  total_venta DECIMAL(10,2),  
  canal_venta VARCHAR(20), -- 'online', 'tienda'  
  segmento_cliente VARCHAR(20)  
);
```

3. Transformaciones necesarias

- Normalización temporal:
Convertir timestamp → DATE estándar (ejemplo: 2025-11-25).
- Cálculo de totales:
 $\text{total_venta} = \text{cantidad} \times \text{precio_unitario}$.
- Enriquecimiento cliente:
Join con CRM para obtener segmento_cliente.
- Homologación de productos:
Crear tabla de referencia productos_dim para mapear códigos distintos entre POS, inventario y web.
- Detección de duplicados:
Usar id_venta como clave primaria y aplicar reglas de deduplicación (ejemplo: ventas repetidas en POS por error de sincronización).

4. Frecuencia y estrategia de carga

- POS (SQL): incremental cada hora → solo nuevas ventas ().
- Inventario (API): carga completa diaria → snapshot de stock.
- CRM (CSV): semanal → actualización de segmentos.
- Web logs (JSON): batch diario → comportamiento online.

5. Ejemplo práctico en Python + Pandas + SQLite

```
import pandas as pd
import sqlite3

# Conexión a SQLite
conn = sqlite3.connect("retail.db")

# --- POS (SQL) ---
ventas_pos = pd.read_sql("SELECT * FROM ventas", conn)
ventas_pos["fecha_venta"] = pd.to_datetime(ventas_pos["timestamp"]).dt.date
ventas_pos["total_venta"] = ventas_pos["cantidad"] * ventas_pos["precio_unitario"]

# --- CRM (CSV) ---
crm = pd.read_csv("clientes.csv")

# --- Web logs (JSON) ---
logs = pd.read_json("logs.json")
logs["fecha_venta"] = pd.to_datetime(logs["timestamp"]).dt.date
logs["canal_venta"] = "online"

# --- Integración ---
ventas = ventas_pos.merge(crm, on="id_cliente", how="left")
ventas = pd.concat([ventas, logs], ignore_index=True)
```

```
# --- Carga a SQLite ---
ventas.to_sql("ventas_consolidadas", conn, if_exists="append", index=False)
```

6. Verificación: resolución de fragmentación

- Fuente heterogénea → esquema unificado:
SQL, API, CSV y JSON se integran en una sola tabla.
- Inconsistencias temporales → fechas normalizadas.
- Datos dispersos de clientes → enriquecimiento con segmento CRM.
- Códigos distintos de producto → tabla de referencia común.
- Duplicados → control con clave primaria y reglas de limpieza.
- Frecuencia adaptada → balance entre frescura y costo de procesamiento.

Conclusión

Este pipeline ETL:

- Centraliza la información dispersa.
- Estandariza formatos y claves.
- Enriquece los datos con contexto de cliente e inventario.
- Optimiza la frecuencia de carga según la naturaleza de cada fuente.

Resultado: una tabla de hechos de ventas consolidada que permite análisis integrados de desempeño, inventario y comportamiento de clientes.

Diagrama Conceptual ETL

